

实验四：众数问题

班级：7 班

姓名：卢瑶

学号：20211018341

● 实验内容

- 给定一个整数序列，每个元素出现的次数称为重数，重数最大的元素称为众数。编写一个程序，对递增有序序列求众数和重数。

● 实验目的

- 锻炼实际问题的算法分析和设计能力。

● 实验要求

- 分析设计完成对应题目算法，并利用头歌平台完成算法实现的调试和测试，取得正确的输出结果。
- 要求所设计的算法时间复杂度至少为 $n\log(n)$ 。
- 按照算法设计的一般过程写出实验报告。

● 实验报告内容应包括以下内容

- 要解决问题的分析描述(输入、输出、过程)；

众数问题

```
#include <stdio.h>
#include <stdlib.h>
//好费空间呀
int* getMode(int a[], int s, int t){ //划分
    int *p = (int*) malloc(sizeof(int) * 2); //自己开辟了内存空间 (堆)
    p[0] = a[s]; //记录枢轴元素
    p[1] = 1; //众数记录 -> 初始化 (相应的重数)
    if(s < t){ //注意加判断条件
        int i = s, j = t; //i -> 首元素, j -> 尾元素
        int tmp = a[s]; //tmp 记录 首元素
        while(i != j){
            while(j > i && a[j] >= tmp){ //从左向右 -> 找到第一个小于a[s]的
                //元素
                if(a[j] == tmp) p[1]++;
                j--; //把 tmp 换成 a[s] 排序会出错? 那里覆
            }
            a[i] = a[j];
            while(i < j && a[i] <= tmp){ //从右向左 -> 找到第一个大于a[s]的
                //元素
            }
        }
    }
}
```

```

        if(a[i] == tmp) p[1]++;
        i++;
    }
    a[j] = a[i]; //一轮快排下来，中间元素的重数便被记录下来了 (p[1] 中)
}
//当 i = j 退出 while 循环
a[i] = tmp; //中间值赋为 tmp
int *p1 = getMode(a, s, i - 1); //问题分解
int *p2 = getMode(a, i + 1, t);
//回溯 || 过程中返回判断
if(p1[1] > p2[1]); //重数比较，最大的作为众数
else
    p1 = p2; //指向同一个数组
if(p[1] > p1[1]);
else
    p = p1; //i 即为划分的“中间界” (意会啦~)
}
return p; //返回存储众数数组的地址 (指针) (数组) [本质理解!]
}

//测试用例: 1 2 2 2 3 3 5 | 1 2 2 3 3 3 5 5 6 6 6 6 6
int main(int argc, char *argv[]) {

    int a[99999];
    int n = 0;

    while(scanf("%d", &a[n])){ //数组未知个数的录入 while 循环的妙用
        n++;
        if(getchar() != ' ')
            break;
    }

    int *p = getMode(a, 0, n-1);

    printf("%d,%d", p[0], p[1]);

}

```

- 算法的设计思路(采用什么算法设计方法，如何划分等)；

利用快速排序的思想，一轮快排后（遍历数组是同时记录重数），进行左右数组的划分，直到最小元素。

○ 算法的描述(用自然语言或流程图或伪代码等方式);

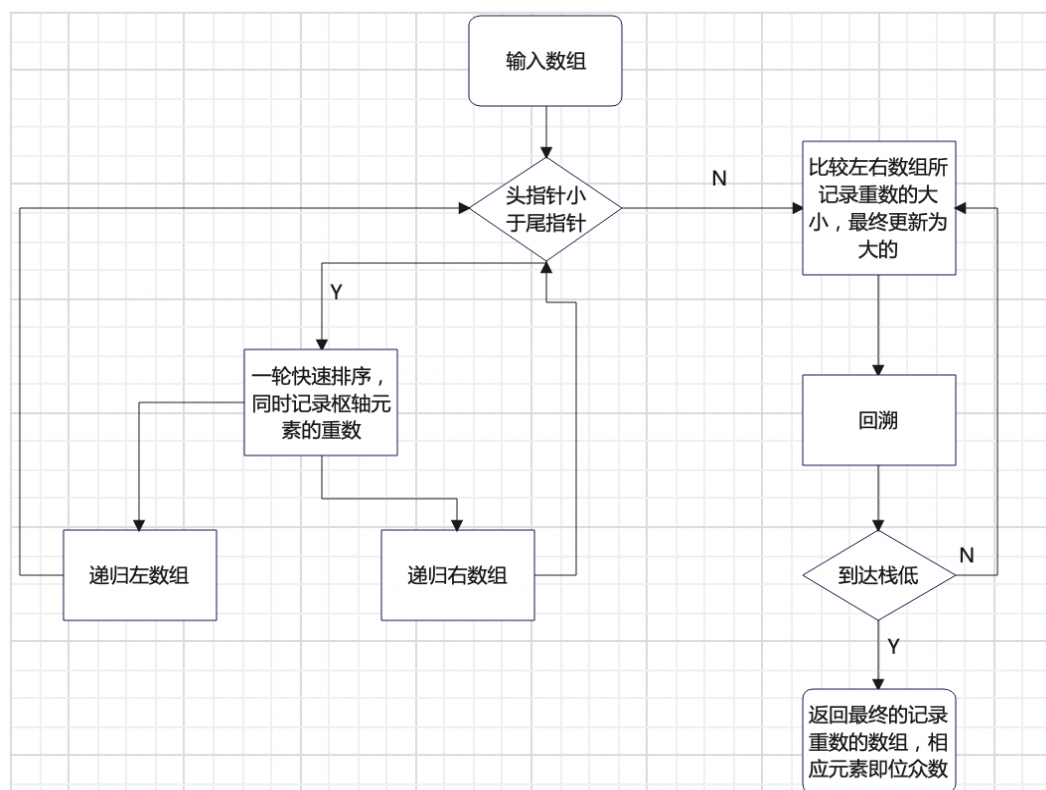


图 1 求众数算法流程图

○ 算法时间和空间效率的分析;

1. 由于是借助快排的思想, 显然算法的时间复杂度为 $O(n \log_2 n)$;
2. 对数组递归的同时, 还要开辟新数组记录相应的重数, 所以空间复杂度为 $O(n^2)$ 。

○ 实验心得(给出你对于蛮力法、分治法等算法设计方法的理解和处理问题的类型的归纳)。

1. 对于时间复杂度可以容忍的问题, 比如元素较少数组的相关操作, 可以采用蛮力法(枚举)
2. 对于一个大问题, 可以向下分解为性质相同的小问题, 比如经典的 Hanoi 问题、求数组中最大最小值问题等, 找到“基例”和“链条”后, 都可以利用分治+递归的方法配合解决, 达到以空间换时间的效果, 从而提高效率。